Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Ingeniería en Ciencias y Sistemas
CURSO: *Manejo e Implementación de Archivos*



MANUAL TECNICO

APLICACIÓN WEB DE GESTIÓN Y SIMULACIÓN DE DISCOS EXT2/EXT3

Nombre: **Carné No.:**

ADILZON ALFREDO VELÁSQUEZ HERNÁNDEZ 201908076

Auxiliar: Williams Bramlley Constanza Oscal

Guatemala, 13 de Octubre 2026

1.1. Objetivo General

Documentar los fundamentos de arquitectura, especificaciones de estructuras de datos binarias, mecanismos de almacenamiento en bajo nivel, protocolo de comunicación API REST y despliegue en infraestructura de computación en la nube correspondientes al sistema C++ DISK ONLINE, proveyendo una guía técnica rigurosa para el mantenimiento, escalamiento y comprensión del software.

1.2. Objetivos Específicos

- Describir la organización física y lógica de los discos virtuales (.mia) y la emulación de sistemas de archivos EXT2 y EXT3 mediante estructuras nativas en lenguaje C++.
- Detallar las fórmulas matemáticas de particionamiento, dimensionamiento del superbloque y cálculo dinámico de estructuras (inodos, bloques y registros de journaling).
- Explicar la arquitectura de microservicios desacoplada entre el motor de persistencia C++ desplegado en AWS EC2 y el cliente interactivo web alojado en AWS S3.
- Documentar la totalidad de instrucciones operativas, validaciones sintácticas y semánticas, y la gestión de permisos UGO sobre el árbol de directorios simulado.

1.3. Alcance del Sistema

El proyecto C++ DISK ONLINE abarca la simulación integral del ciclo de vida del almacenamiento en disco secundario: desde la partición a bajo nivel hasta la navegación jerárquica por interfaz gráfica. Sus alcances contemplan:

- Gestión de almacenamiento primario, extendido y lógico con algoritmos de ajuste First Fit (FF), Best Fit (BF) y Worst Fit (WF).
- Formateo lógico completo (Full) y rápido (Fast) bajo estándares EXT2 (sin bitácora) y EXT3 (con bitácora de transacciones de tamaño fijo).
- Sistema de autenticación y seguridad multinivel mediante archivo users.txt y permisos de lectura, escritura y ejecución (rwx).
- Exposición de un servidor HTTP embebido de alto rendimiento en C++ con soporte para Cross-Origin Resource Sharing (CORS).
- Interfaz gráfica web responsiva con terminal de comandos integrada, explorador de directorios, visor de contenido, tabla de journaling y renderizado gráfico de reportes.

1.4. Tecnologías y Herramientas Utilizadas

Tecnología / Herramienta	Función en el Sistema
C++17 / GCC (g++)	Lenguaje de programación para la lógica central del sistema de archivos, manipulación de archivos binarios y gestión de memoria.
cpp-httplib	Biblioteca ligera de C++ ('header-only') utilizada para implementar el servidor HTTP REST embebido y el enrutamiento de peticiones.
nlohmann/json	Biblioteca moderna de C++ para la serialización y deserialización de objetos JSON en el intercambio cliente-servidor.
GNU Make	Herramienta de automatización de compilación para la generación de objetos y binarios ejecutables.
Graphviz (dot)	Generador gráfico por línea de comandos para la renderización de reportes de disco, árboles de inodos y bloques.
[REACT / ANGULAR / VUE]	[COMPLETAR: Framework frontend seleccionado y versión utilizada] para la construcción de la interfaz gráfica de usuario.

Vite / Build Tools	Herramienta de compilación y empaquetado para la generación del bundle estático web.
Axios	Cliente HTTP basado en promesas para la comunicación asíncrona entre el navegador y el servidor REST.
Amazon Web Services (AWS)	Infraestructura cloud utilizada para el alojamiento del backend (EC2 Ubuntu) y frontend (S3 Static Hosting).

2. DESCRIPCIÓN DE LA ARQUITECTURA DEL SISTEMA

2.1. Arquitectura General Frontend / Backend

El sistema se basa en una arquitectura cliente-servidor desacoplada. El frontend opera como una aplicación de una sola página (SPA), encargada de la captura de comandos, selección de discos y particiones, y visualización reactiva del sistema de archivos. El backend actúa como un motor de procesamiento determinístico sin estado en memoria persistente, donde cada operación se valida y se plasma en tiempo real sobre los archivos binarios que representan los discos virtuales (.mia).

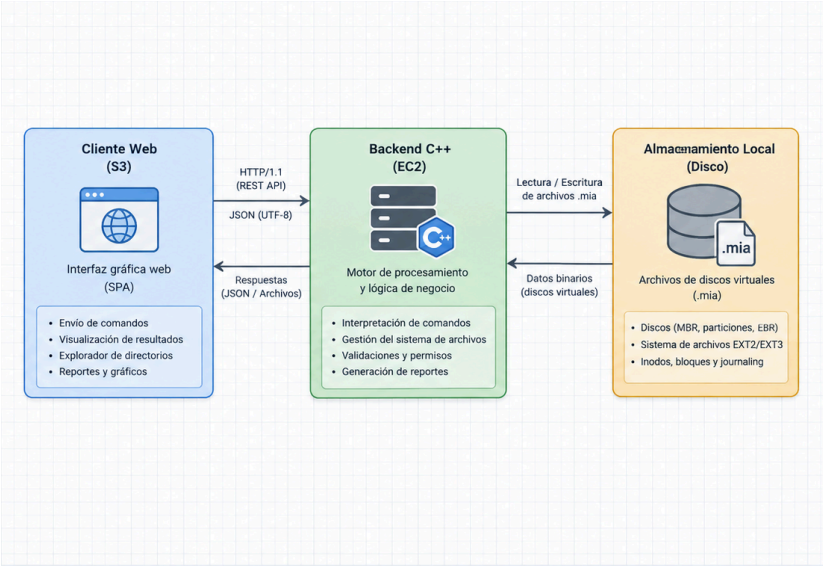


Figura 2.1. Diagrama de arquitectura lógica del sistema C++ DISK ONLINE.

2.2. Protocolo de Comunicación y Formato de Datos

La comunicación inter-componentes se establece a través del protocolo HTTP/1.1 mediante peticiones asíncronas REST. Todas las cargas útiles ('payloads') enviadas y recibidas utilizan codificación UTF-8 en formato JSON estándar. Para la transferencia de reportes generados como imágenes o diagramas, se provee soporte de descarga binaria con cabeceras MIME dinámicas.

Método	Endpoint	Descripción Operativa	Cuerpo Solicitud	Cuerpo Respuesta
POST	/api/execute	Envía una cadena de texto con uno o varios comandos para su interpretación secuencial.	{ "commands": "mkdisk..." }	{ "status": "success", "output": "..." }
GET	/api/file	Retorna el flujo de bytes de un archivo o reporte generado en el servidor.	Query param: path	Archivo binario / texto plano según MIME
POST	/api/login	Valida las credenciales de usuario y abre sesión en la partición especificada.	{ "user": "root", "pass": "123", "id": "..." }	{ "status": "success", "isLoggedIn": true, ... }
POST	/api/logout	Cierra la sesión activa actual del sistema.	Vacío	{ "status": "success", "isLoggedIn": false }
GET	/api/session	Consulta el estado actual de la sesión en el servidor.	Vacío	{ "isLoggedIn": bool, "user": string, "id": string }
GET	/api/partitions	Retorna la lista de todas las particiones reconocidas y su estado de montaje.	Query param opcional: disk	{ "status": "success", "partitions": [...] }
GET	/api/disks	Lista los discos virtuales detectados en el servidor junto con su MBR.	Query param opcional: path	{ "status": "success", "disks": [...] }
GET	/api/journal	Obtiene el listado cronológico de transacciones en la bitácora EXT3.	Query param: id	{ "status": "success", "journaling": [...] }

GET	/api/tree	Permite la navegación jerárquica de carpetas o lectura de archivos individuales.	Query params: id, path, output	{ "status": "success", "type": "folder|file", ... }

```
   // Helper para capturar stdout de comandos
   auto run_captured = [&](const std::function<void()>& func) -> string {
       stringstream ss;
       streambuf* prev_cout = cout.rdbuf(ss.rdbuf());
       try {
           func();
       } catch (const exception& e) {
           cout << "\n[EXCEPCION]: " << e.what() << endl;
       } catch (...) {
           cout << "\n[ERROR DESCONOCIDO]" << endl;
       }
       cout.rdbuf(prev_cout);
       return ss.str();
   };


   // ------------------------------------------------------------
   // 1. POST /api/execute
   // ------------------------------------------------------------
   svr.Post("/api/execute", [&](const httplib::Request &req,
httplib::Response &res) {
       set_cors_headers(res);
       string commands = "";
```

```
        if (req.has_header("Content-Type") &&
req.get_header_value("Content-Type").find("application/json") !=
string::npos) {
            try {
                auto body = json::parse(req.body);
                if (body.contains("commands") &&
body["commands"].is_string()) {
                    commands = body["commands"].get<string>();
                } else if (body.contains("command") &&
body["command"].is_string()) {
                    commands = body["command"].get<string>();
                } else if (body.contains("code") &&
body["code"].is_string()) {
                    commands = body["code"].get<string>();
                }
            } catch (...) {
                commands = req.body;
            }
        } else {
            commands = req.body;
        }


        if (commands.empty()) {
            json err_resp = {{"status", "error"}, {"message", "No se
recibieron comandos para ejecutar"}};
            res.set_content(err_resp.dump(), "application/json");
            res.status = 400;
            return;
        }
```

```
    string output = run_captured([&]() {
        scan.execute_script(commands);
    });

    json ok_resp = {
        {"status", "success"},
        {"output", output}
    };
    res.set_content(ok_resp.dump(), "application/json");
});
```

Listado 2.1. Declaración de endpoints REST y captura de salida estándar en C++.

2.3. Arquitectura del Despliegue en Amazon Web Services (AWS)

La solución se encuentra implementada en la nube comercial de AWS para garantizar accesibilidad remota y emulación realista de entornos productivos distribuidos.

2.3.1. AWS S3 (Alojamiento Web Estático)

El cliente web se compila a artefactos estáticos (HTML, JS, CSS, assets) y se transfiere a un bucket de Amazon S3 configurado en modo Static Website Hosting:

- Nombre del bucket: [COMPLETAR: Nombre del bucket en AWS S3]
- Configuración de alojamiento: Habilitado con index.html como documento de índice y documento de error.
- Control de acceso: Desactivación de 'Bloqueo de acceso público' (Block Public Access) para permitir la lectura pública de los artefactos web.
- Política del bucket (Bucket Policy): Se adjunta una política JSON que concede el permiso s3:GetObject a la entidad anónima (*).

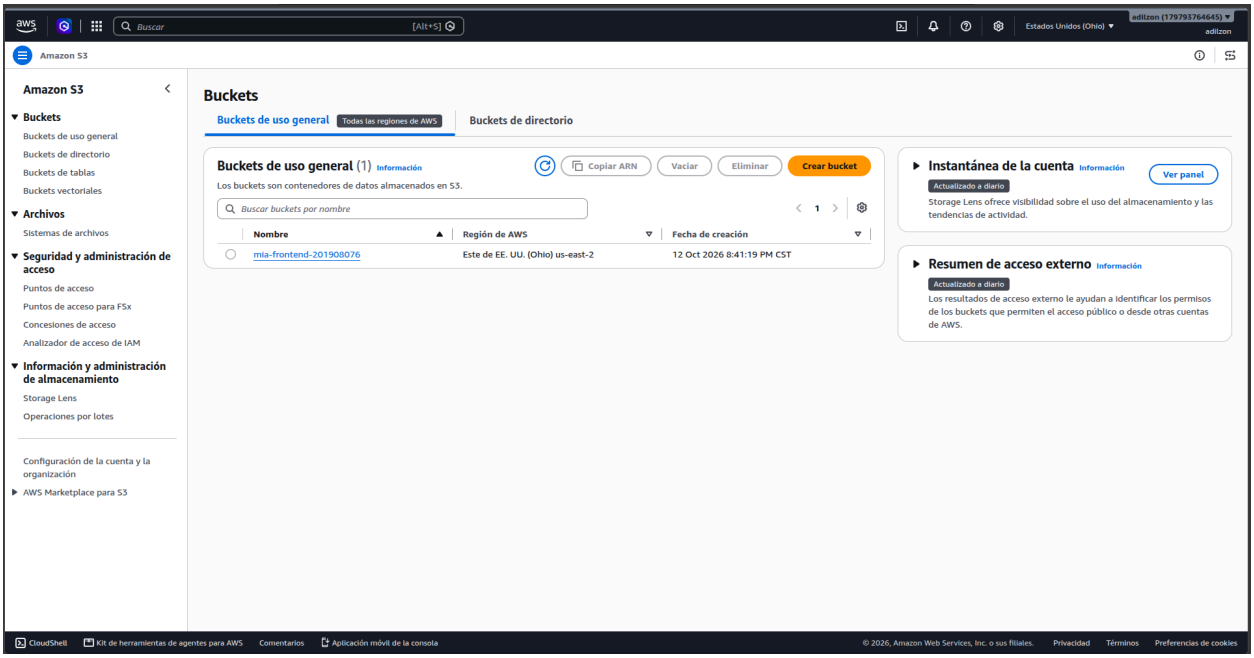
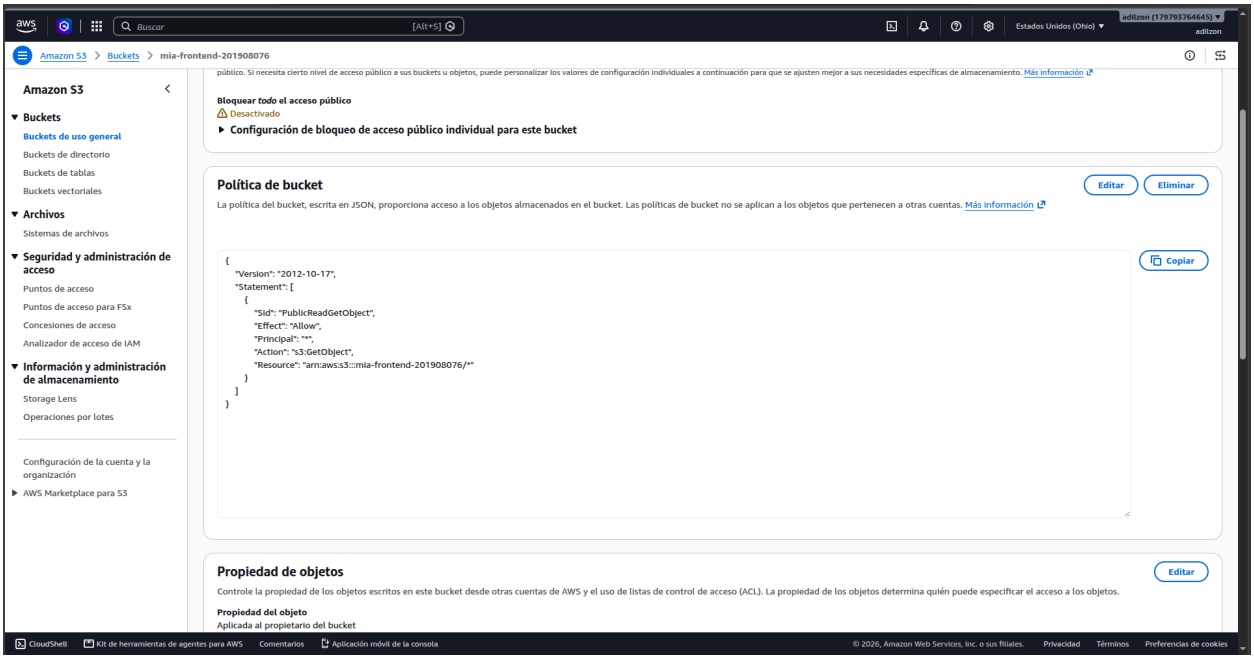


Figura 2.2. Panel de configuración de alojamiento estático en Amazon S3.



Listado 2.2. Política de acceso público de solo lectura para el bucket S3.

2.3.2. AWS EC2 (Servidor Backend en Ubuntu)

El motor C++ se aloja en una instancia virtual de cómputo Amazon EC2 ejecutando el sistema operativo Linux Ubuntu Server:

- Tipo de instancia: [COMPLETAR: Tipo de instancia, ej. t2.micro]
- Sistema Operativo: Ubuntu Server [COMPLETAR: Versión, ej. 22.04 LTS]
- Dirección IP Pública / DNS: [COMPLETAR: IP pública o dominio de la instancia EC2]

- Grupo de Seguridad (Security Group): Reglas de entrada (Inbound Rules) configuradas para admitir:
- - Puerto 22 (TCP): Acceso por SSH restringido para administración remota.
- - Puerto 80 / 443 (TCP): Tráfico web general.
- - Puerto [COMPLETAR: Puerto del backend, ej. 8080] (TCP): Tráfico hacia el servidor de API C++.
- Mecanismo de persistencia del proceso: [COMPLETAR: nohup / systemd service / tmux] para garantizar la ejecución desatendida del binario api_server.

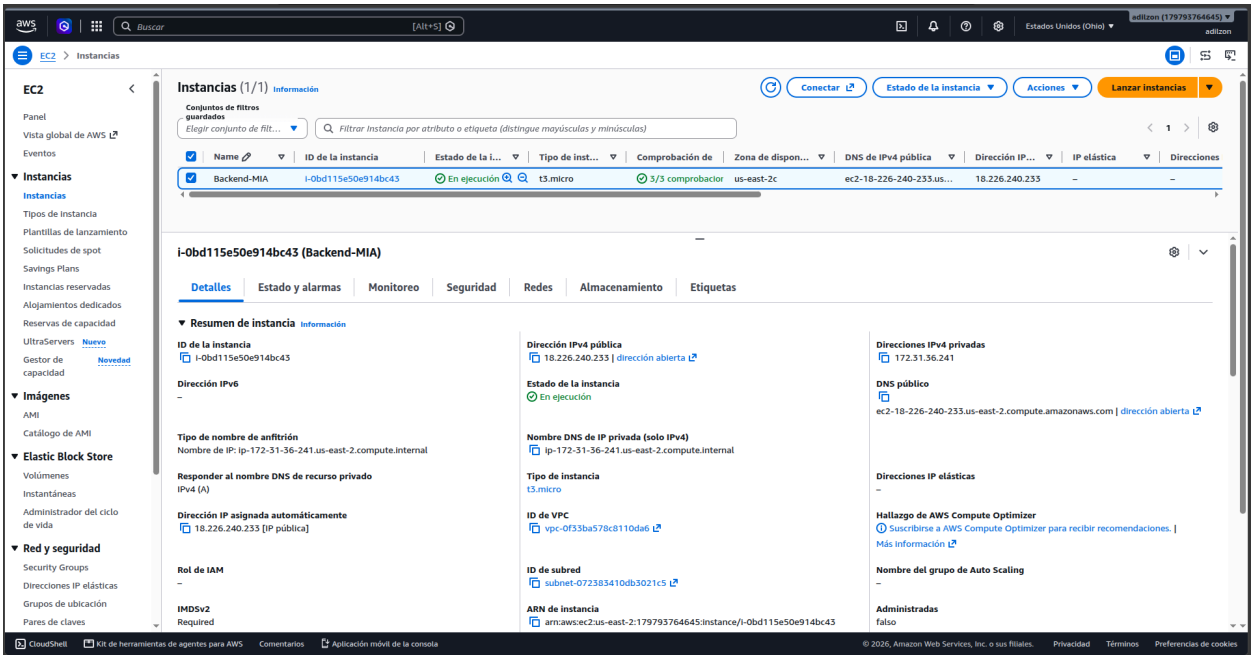


Figura 2.3. Configuración de instancia EC2 y Security Group en AWS.

2.3.3. Conexión S3–EC2 y Configuración de CORS

Dado que el frontend (servido desde el dominio de S3) y el backend (servido desde la IP pública de EC2 en un puerto específico) poseen orígenes distintos, los navegadores web ejecutan el mecanismo de seguridad Same-Origin Policy. Para permitir la comunicación:

- El backend implementa un manejador global de solicitudes 'preflight' (OPTIONS) que responde con código de estado 204 No Content.
- Todas las respuestas HTTP incluyen las cabeceras requeridas:
- - Access-Control-Allow-Origin: * (o el dominio específico del bucket S3)
- - Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
- - Access-Control-Allow-Headers: Content-Type, Authorization, X-Requested-With
- - Access-Control-Max-Age: 86400

```
// ------------------------------------------------------------
// Configuración Global de CORS
// ------------------------------------------------------------
auto set_cors_headers = [](httplib::Response &res) {
    // En httplib, res.headers es un std::unordered_multimap.
```

```
        // Llamar set_header() múltiples veces inserta duplicados en vez de sobrescribir.
        // Para asegurar que cada cabecera aparezca EXACTAMENTE UNA VEZ, borramos cualquier previa:
        res.headers.erase("Access-Control-Allow-Origin");
        res.headers.erase("Access-Control-Allow-Methods");
        res.headers.erase("Access-Control-Allow-Headers");
        res.headers.erase("Access-Control-Max-Age");

        res.set_header("Access-Control-Allow-Origin", "*");
        res.set_header("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTIONS");
        res.set_header("Access-Control-Allow-Headers", "Content-Type, Authorization, X-Requested-With");
        res.set_header("Access-Control-Max-Age", "86400");
    };

    // Preflight Handler para todas las rutas (OPTIONS)
    svr.Options(R"(.*)", [set_cors_headers](const httplib::Request &, httplib::Response &res) {
        set_cors_headers(res);
        res.status = 204;
    });

    // Interceptor pre-routing para solicitudes OPTIONS
    svr.set_pre_routing_handler([set_cors_headers](const httplib::Request &req, httplib::Response &res) {
        if (req.method == "OPTIONS") {
            set_cors_headers(res);
            res.status = 204;
            return httplib::Server::HandlerResponse::Handled;
        }
        return httplib::Server::HandlerResponse::Unhandled;
    });
```

```
   // Interceptor post-routing: garantiza cabeceras CORS en todas las
respuestas sin duplicar
   svr.set_post_routing_handler([set_cors_headers](const
httplib::Request &, httplib::Response &res) {
       set_cors_headers(res);
   });
```

Listado 2.3. Interceptor de cabeceras CORS en cpp-httplib.

3. EXPLICACIÓN DE LAS ESTRUCTURAS DE DATOS

El sistema implementa el modelo de almacenamiento basado en bloques e inodos propio de los sistemas de archivos de la familia Unix. Todas las estructuras se encuentran modeladas mediante estructuras ('structs') en C++ con empaquetamiento directo y tipos de longitud fija para su escritura y lectura binaria sin pérdida de alineación.

3.1. Master Boot Record (MBR)

Estructura principal ubicada en el byte 0 de cada archivo binario .mia. Almacena la tabla de particiones primarias y los metadatos generales del disco virtual.

Campo	Tipo de Dato	Descripción Funcional
mbr_tamano	[COMPLETAR: int (4 bytes)]	Tamaño total del disco virtual expresado en bytes.
mbr_fecha_creacion	[COMPLETAR: time_t (8 bytes)]	Timestamp que registra la fecha y hora exacta de creación del disco.
mbr_disk_signature	[COMPLETAR: int (4 bytes)]	Firma numérica aleatoria que identifica unívocamente al disco.
disk_fit	[COMPLETAR: char (1 byte)]	Tipo de ajuste global del disco: 'F' (First), 'B' (Best) o 'W' (Worst).
mbr_Partition_1	[COMPLETAR: Partition (struct)]	Primera entrada de partición en la tabla del disco.
mbr_Partition_2	[COMPLETAR: Partition (struct)]	Segunda entrada de partición en la tabla del disco.

mbr_Partition_3	[COMPLETAR: Partition (struct)]	Tercera entrada de partición en la tabla del disco.
mbr_Partition_4	[COMPLETAR: Partition (struct)]	Cuarta entrada de partición en la tabla del disco.

3.2. Partition (Partición Primaria / Extendida)

Representa una subdivisión contigua dentro del espacio direccionable del disco virtual.

Campo	Tipo de Dato	Descripción Funcional
part_status	[COMPLETAR: char (1 byte)]	Estado de la partición: '0' inactiva/disponible, '1' asignada/activa.
part_type	[COMPLETAR: char (1 byte)]	Tipo de partición: 'P' (Primaria), 'E' (Extendida).
part_fit	[COMPLETAR: char (1 byte)]	Ajuste de partición: 'F' (First Fit), 'B' (Best Fit), 'W' (Worst Fit).
part_start	[COMPLETAR: int (4 bytes)]	Dirección en bytes donde inicia la partición dentro del archivo .mia.
part_size	[COMPLETAR: int (4 bytes)]	Capacidad total de la partición en bytes.
part_name	[COMPLETAR: char[16] (16 bytes)]	Nombre alfanumérico identificador de la partición (máximo 16 caracteres).

3.3. Extended Boot Record (EBR)

Estructura utilizada para encadenar particiones lógicas dentro del espacio delimitado por una partición extendida. Actúa como una lista enlazada simple ubicada en el almacenamiento secundario.

Campo	Tipo de Dato	Descripción Funcional
part_status	[COMPLETAR: char (1 byte)]	Estado de la partición lógica: '0' inactiva/eliminada, '1' activa.

part_fit	[COMPLETAR: char (1 byte)]	Tipo de ajuste de asignación de la partición lógica ('F', 'B', 'W').
part_start	[COMPLETAR: int (4 bytes)]	Byte del archivo .mia donde inicia el contenido de esta partición lógica.
part_size	[COMPLETAR: int (4 bytes)]	Tamaño útil de la partición lógica en bytes.
part_next	[COMPLETAR: int (4 bytes)]	Byte de inicio del siguiente EBR en la lista enlazada (-1 si es el último).
part_name	[COMPLETAR: char[16] (16 bytes)]	Nombre identificador de la partición lógica.

3.4. Superblock (Superbloque EXT2 / EXT3)

Estructura central del sistema de archivos. Contiene la geometría, contadores de uso y los punteros a las direcciones base de los bitmaps, inodos y bloques.

Campo	Tipo de Dato	Descripción Funcional
s_filesystem_type	[COMPLETAR: int]	Identificador de la versión del sistema de archivos: 2 (EXT2), 3 (EXT3).
s_inodes_count	[COMPLETAR: int]	Número total de inodos asignados a la partición.
s_blocks_count	[COMPLETAR: int]	Número total de bloques asignados a la partición (3 * n).
s_free_inodes_count	[COMPLETAR: int]	Cantidad actual de inodos disponibles sin asignar.
s_free_blocks_count	[COMPLETAR: int]	Cantidad actual de bloques disponibles sin asignar.
s_mtime	[COMPLETAR: time_t]	Fecha y hora del último montaje del sistema de archivos.
s_umtime	[COMPLETAR: time_t]	Fecha y hora del último desmontaje del sistema de archivos.

s_mnt_count	[COMPLETAR: int]	Contador acumulado de veces que ha sido montada la partición.
s_magic	[COMPLETAR: int]	Valor identificador del sistema de archivos (0xEF53).
s_inode_size	[COMPLETAR: int]	Tamaño en bytes de la estructura de inodo (sizeof(Inodes)).
s_block_size	[COMPLETAR: int]	Tamaño en bytes de los bloques de almacenamiento (sizeof(Folderblock)).
s_fist_ino	[COMPLETAR: int]	Índice del primer inodo disponible en el bitmap.
s_first_blo	[COMPLETAR: int]	Índice del primer bloque disponible en el bitmap.
s_bm_inode_start	[COMPLETAR: int]	Posición en bytes donde inicia el bitmap de inodos.
s_bm_block_start	[COMPLETAR: int]	Posición en bytes donde inicia el bitmap de bloques.
s_inode_start	[COMPLETAR: int]	Posición en bytes donde inicia la tabla de inodos.
s_block_start	[COMPLETAR: int]	Posición en bytes donde inicia la tabla de bloques.

3.5. Tabla de Inodos (Inodes)

Estructura que describe cada objeto (archivo o carpeta) existente en el sistema de archivos, sin incluir su nombre.

Campo	Tipo de Dato	Descripción Funcional
i_uid	[COMPLETAR: int]	Identificador numérico del usuario propietario del archivo/carpeta.
i_gid	[COMPLETAR: int]	Identificador numérico del grupo propietario del archivo/carpeta.

i_size	[COMPLETAR: int]	Tamaño actual del archivo expresado en bytes (0 para carpetas).
i_atime	[COMPLETAR: time_t]	Fecha y hora del último acceso de lectura al inodo.
i_ctime	[COMPLETAR: time_t]	Fecha y hora de creación del inodo.
i_mtime	[COMPLETAR: time_t]	Fecha y hora de la última modificación en el contenido del inodo.
i_block[15]	[COMPLETAR: int[15]]	Arreglo de 15 apuntadores a bloques: 12 directos, 1 indirecto simple, 1 indirecto doble y 1 indirecto triple.
i_type	[COMPLETAR: char]	Tipo de inodo: '0' para carpeta (directorio), '1' para archivo de datos.
i_perm	[COMPLETAR: int]	Permisos de acceso codificados en notación octal UGO (ej. 664).

3.6. Bloques de Almacenamiento

Unidades atómicas de almacenamiento de datos en la partición (tamaño homogéneo de 64 bytes):

- Bloque de Carpetas (Folderblock): Contiene 4 estructuras Content b_content[4]. Cada entrada incluye b_name[12] (nombre del archivo o subcarpeta) y b_inodo (número de inodo al que apunta, o -1 si está vacía).
- Bloque de Archivos (Fileblock): Contiene un arreglo de caracteres b_content[64] con el texto o bytes brutos del archivo.
- Bloque de Apuntadores (Pointerblock): Contiene un arreglo b_pointers[16] de números enteros, permitiendo referenciar hasta 16 bloques adicionales para direccionamiento indirecto.

3.7. Estructuras de Bitácora: Journal e Information

Estructuras exclusivas de sistemas EXT3 para el registro secuencial de operaciones atómicas antes de su confirmación física en disco:

Campo	Tipo de Dato	Descripción Funcional

j_count	[COMPLETAR: int]	Número secuencial de transacción en el journal.
j_content.i_operation	[COMPLETAR: char[10]]	Comando ejecutado ('mkdir', 'mkfile', etc.).
j_content.i_path	[COMPLETAR: char[32]]	Ruta absoluta del sistema de archivos afectada.
j_content.i_content	[COMPLETAR: char[64]]	Contenido o datos asociados a la transacción.
j_content.i_date	[COMPLETAR: float / time_t]	Timestamp que registra el momento de ejecución.

3.8. Distribución de la Partición en EXT2 y EXT3

La memoria secundaria dentro del archivo binario .mia se organiza secuencialmente de la siguiente forma:

Distribución EXT2:

- [1] Superblock (Tamaño fijo: sizeof(Superblock))
- [2] Bitmap de Inodos (Tamaño: n bytes)
- [3] Bitmap de Bloques (Tamaño: 3n bytes)
- [4] Tabla de Inodos (Tamaño: n * sizeof(Inodes))
- [5] Tabla de Bloques (Tamaño: 3n * sizeof(Folderblock))

Distribución EXT3:

- [1] Superblock (Tamaño fijo: sizeof(Superblock))
- [2] Journaling (Tamaño fijo: 50 * sizeof(Journal))
- [3] Bitmap de Inodos (Tamaño: n bytes)
- [4] Bitmap de Bloques (Tamaño: 3n bytes)
- [5] Tabla de Inodos (Tamaño: n * sizeof(Inodes))
- [6] Tabla de Bloques (Tamaño: 3n * sizeof(Folderblock))

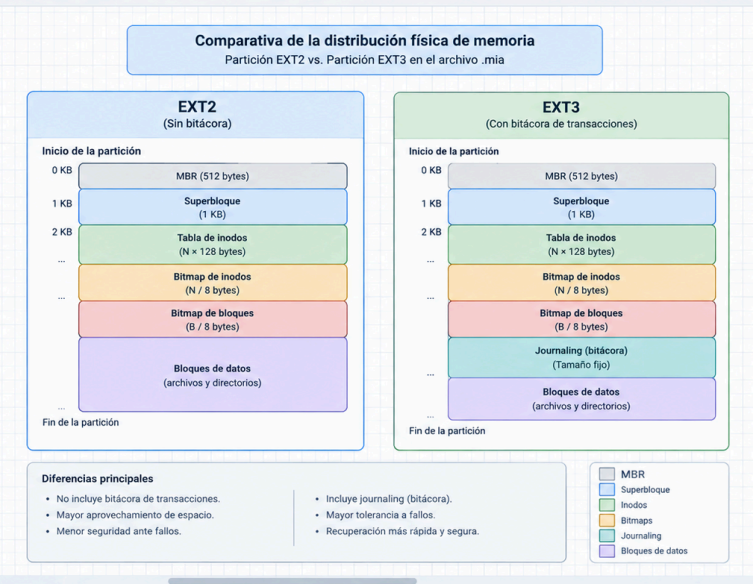


Figura 3.1. Mapa de distribución física de estructuras en particiones EXT2 y EXT3.

3.9. Fórmula del Cálculo de 'n' para EXT3

El número 'n' representa la cantidad máxima de inodos que pueden albergarse en la partición, guardando una proporción de 1 inodo por cada 3 bloques de datos. En EXT3, se deduce a partir de la ecuación del tamaño total de la partición:

tamaño_partición = sizeof(Superblock) + 50 * sizeof(Journaling) + n + 3*n + n * sizeof(Inodo) + 3*n * sizeof(Bloque)

Factorizando la variable n:

tamaño_partición - sizeof(Superblock) - 50 * sizeof(Journaling) = n * (1 + 3 + sizeof(Inodo) + 3 * sizeof(Bloque))

Por lo tanto, la fórmula computacional aplicada es:

n = floor((tamaño_partición - sizeof(Superblock) - 50 * sizeof(Journal)) / (4 + sizeof(Inodo) + 3 * sizeof(Bloque)))

Ejemplo de cálculo con valores numéricos:

- Tamaño de partición de prueba: 2,097,152 bytes (2 MB)
- sizeof(Superblock): [COMPLETAR: Valor en bytes, ej. 80 / 84 bytes]
- sizeof(Journal): [COMPLETAR: Valor en bytes, ej. 112 / 120 bytes]
- 50 * sizeof(Journal): [COMPLETAR: Valor total del journaling en bytes]
- sizeof(Inodo): [COMPLETAR: Valor en bytes, ej. 104 / 128 bytes]
- sizeof(Bloque): [COMPLETAR: Valor en bytes, ej. 64 bytes]
- Denominador = 4 + sizeof(Inodo) + 3 * sizeof(Bloque) = [COMPLETAR: Valor del denominador]
- Resultado de n: [COMPLETAR: Valor numérico entero resultante de floor(n)]

4. DESCRIPCIÓN DE LOS COMANDOS IMPLEMENTADOS

A continuación se documenta el repertorio completo de comandos implementados en el intérprete del sistema, detallando sus argumentos, validaciones de integridad y efectos binarios sobre las estructuras de datos.

4.1. Comando MKDISK

Descripción: Crea un archivo binario (.mia o .dsk) que emula un disco rígido virtual, rellenándolo de ceros y grabando el MBR inicial.

Parámetro	Condición	Descripción
size	Obligatorio	Entero positivo mayor a 0 que indica la capacidad del disco.
path	Obligatorio	Ruta de archivo del disco virtual en el sistema anfitrión.
fit	Opcional	Ajuste de colocación: 'BF' (Best), 'FF' (First), 'WF' (Worst). Por defecto 'FF'.
unit	Opcional	Unidades de tamaño: 'K' (Kilobytes) o 'M' (Megabytes). Por defecto 'M'.

Ejemplo de uso:
mkdisk -size=10 -u=M -path=/home/user/disco1.mia -f=FF

Efecto sobre estructuras internas: Crea el archivo binario, escribe caracteres nulos en todo el tamaño y escribe la cabecera MBR al inicio con la firma aleatoria generada.

Validaciones y control de errores: Verifica que el tamaño sea positivo y que la ruta sea válida en el sistema anfitrión. Error si el archivo ya existe.

4.2. Comando RMDISK

Descripción: Elimina de forma irreversible un archivo de disco virtual del sistema anfitrión.

Parámetro	Condición	Descripción
path	Obligatorio	Ruta completa del archivo de disco a eliminar.

Ejemplo de uso:
rmdisk -path=/home/user/disco1.mia

Efecto sobre estructuras internas: Invoca la llamada al sistema de eliminación de archivos (remove) destruyendo todos los datos del disco.

Validaciones y control de errores: Valida la existencia del archivo previo a la eliminación. Solicita confirmación en consola interactiva.

4.3. Comando FDISK

Descripción: Administra las particiones del disco: creación de primarias, extendidas y lógicas, eliminación y redimensionamiento.

Parámetro	Condición	Descripción
size	Obligatorio (creación)	Capacidad de la partición a crear.
path	Obligatorio	Ruta del disco virtual.
name	Obligatorio	Nombre único de la partición en el disco.
unit	Opcional	Unidades: 'B' (Bytes), 'K' (Kilobytes), 'M' (Megabytes). Por defecto 'K'.
type	Opcional	'P' (Primaria), 'E' (Extendida), 'L' (Lógica). Por defecto 'P'.
fit	Opcional	'BF', 'FF', 'WF'. Por defecto 'WF'.
delete	Opcional	'Full' para marcar la partición como libre o rellenarla.
add	Opcional	Cantidad positiva o negativa de espacio a redimensionar.

Ejemplo de uso:
fdisk -size=3 -u=M -path=/home/user/disco1.mia -type=P -name=Particion1

Efecto sobre estructuras internas: Modifica el MBR actualizando una de las 4 entradas Partition, o navega por los EBR para crear/alterar particiones lógicas.

Validaciones y control de errores: No permite más de 4 particiones primarias/extendidas combinadas, ni más de 1 extendida por disco. Valida que no haya colisiones de espacio ni nombres repetidos.

4.4. Comando MOUNT

Descripción: Carga una partición del disco en memoria volátil de la aplicación, asignándole un identificador alfanumérico único para su uso operativo.

Parámetro	Condición	Descripción
path	Obligatorio	Ruta del archivo de disco.
name	Obligatorio	Nombre de la partición a montar.

Ejemplo de uso:
mount -path=/home/user/disco1.mia -name=Particion1

Efecto sobre estructuras internas: Inserta la partición en la tabla de montajes asignándole un ID formado por el carnet del estudiante, número correlativo de disco y letra correlativa de partición (ej. 761A).

Validaciones y control de errores: Valida que la partición exista, no esté previamente montada y que sea de tipo Primaria o Lógica.

4.5. Comando MOUNTED

Descripción: Despliega el listado completo de particiones actualmente montadas en el sistema con sus identificadores y rutas.

Ejemplo de uso:
mounted

Efecto sobre estructuras internas: Lee el arreglo o estructura de montajes en memoria y formatea los datos en salida estándar.

Validaciones y control de errores: Si no hay particiones montadas, despliega un mensaje informativo indicándolo.

4.6. Comando UNMOUNT

Descripción: Descarga una partición montada liberando su identificador asignado.

Parámetro	Condición	Descripción
id	Obligatorio	Identificador único asignado previamente con mount (ej. 761A).

Ejemplo de uso:
unmount -id=761A

Efecto sobre estructuras internas: Marca la entrada de montaje como inactiva en la tabla de memoria.

Validaciones y control de errores: Verifica que el identificador corresponda a una partición efectivamente montada en la sesión actual.

4.7. Comando MKFS

Descripción: Formatea una partición previamente montada inicializando el sistema de archivos EXT2 o EXT3.

Parámetro	Condición	Descripción
id	Obligatorio	Identificador de la partición montada a formatear.
type	Opcional	Nivel de formateo: 'Full' o 'Fast'. Por defecto 'Full'.
fs	Opcional	Versión del sistema de archivos: '2fs' (EXT2) o '3fs' (EXT3). Por defecto '2fs'.

Ejemplo de uso:
mkfs -id=761A -type=full -fs=3fs

Efecto sobre estructuras internas: Calcula n, escribe el Superbloque, bitácora (si es EXT3), bitmaps en '0', inodos vacíos, bloques vacíos, e inicializa el inodo 0 (raíz '/') y el inodo 1 con el archivo '/users.txt'.

Validaciones y control de errores: Valida que la partición esté montada y que su tamaño sea suficiente para albergar las estructuras mínimas (n > 2).

4.8. Comando LOGIN

Descripción: Inicia una sesión de usuario en una partición formateada, permitiendo acceder a comandos restringidos por permisos.

Parámetro	Condición	Descripción
usr	Obligatorio	Nombre de usuario registrado en users.txt.
pwd	Obligatorio	Contraseña del usuario.
id	Obligatorio	Identificador de la partición donde se inicia sesión.

Ejemplo de uso:
login -usr=root -pwd=123 -id=761A

Efecto sobre estructuras internas: Lee el archivo /users.txt del disco, valida usuario, contraseña y estado activo, y almacena las credenciales en la estructura global de sesión.

Validaciones y control de errores: Deniega el acceso si ya existe una sesión abierta, si las credenciales son erróneas o si el usuario fue eliminado lógicamente.

4.9. Comando LOGOUT

Descripción: Finaliza la sesión del usuario actual en el sistema de archivos.

Ejemplo de uso:
logout

Efecto sobre estructuras internas: Restablece la estructura de sesión activa, limpiando UID, GID y credenciales en memoria.

Validaciones y control de errores: Retorna error si se invoca sin que exista una sesión autenticada previamente.

4.10. Comando MKGRP

Descripción: Crea un nuevo grupo de usuarios en el sistema de archivos.

Parámetro	Condición	Descripción
name	Obligatorio	Nombre del nuevo grupo (máximo 10 caracteres).

Ejemplo de uso:
mkgrp -name=usuarios

Efecto sobre estructuras internas: Abre /users.txt, calcula el siguiente correlativo numérico de grupo y añade la línea 'GID,G,nombre'. Si es EXT3, registra la acción en el Journaling.

Validaciones y control de errores: Exige sesión iniciada con privilegios de usuario 'root'. Falla si el grupo ya existe o excede la longitud máxima.

4.11. Comando RMGRP

Descripción: Elimina lógicamente un grupo de usuarios existente.

Parámetro	Condición	Descripción

name	Obligatorio	Nombre del grupo a remover.

Ejemplo de uso:
rmgrp -name=usuarios

Efecto sobre estructuras internas: Modifica la entrada en /users.txt sustituyendo el GID por 0 (ej. '0,G,usuarios'). Registra la transacción en el Journaling si es EXT3.

Validaciones y control de errores: Solo permitido para 'root'. Falla si el grupo no existe o ya se encuentra eliminado con GID 0.

4.12. Comando MKUSR

Descripción: Registra un nuevo usuario asociándolo a un grupo previamente creado.

Parámetro	Condición	Descripción
usr	Obligatorio	Nombre de usuario único (máximo 10 caracteres).
pwd	Obligatorio	Contraseña del usuario (máximo 10 caracteres).
grp	Obligatorio	Nombre de un grupo activo existente.

Ejemplo de uso:
mkusr -usr=operador -pwd=pass123 -grp=usuarios

Efecto sobre estructuras internas: Añade la línea 'UID,U,grupo,usuario,contraseña' en /users.txt. Registra en Journaling si es EXT3.

Validaciones y control de errores: Exclusivo para 'root'. Falla si el usuario ya existe, el grupo no existe o si los campos superan la longitud permitida.

4.13. Comando RMUSR

Descripción: Elimina lógicamente un usuario del sistema.

Parámetro	Condición	Descripción
usr	Obligatorio	Nombre del usuario a eliminar.

Ejemplo de uso:
rmusr -usr=operador

Efecto sobre estructuras internas: Modifica la línea en /users.txt asignándole UID 0 (ej. '0,U,grupo,usuario,pwd'). Registra en Journaling si es EXT3.

Validaciones y control de errores: Exclusivo para 'root'. Falla si el usuario no existe, ya estaba eliminado o si se intenta eliminar al usuario 'root'.

4.14. Comando CHGRP

Descripción: Cambia el grupo asignado a un usuario existente.

Parámetro	Condición	Descripción
usr	Obligatorio	Nombre del usuario al que se cambiará el grupo.
grp	Obligatorio	Nombre del nuevo grupo de destino.

Ejemplo de uso:
chgrp -usr=operador -grp=administradores

Efecto sobre estructuras internas: Reescribe la línea del usuario en /users.txt actualizando el campo de grupo. Registra en Journaling si es EXT3.

Validaciones y control de errores: Exclusivo para 'root'. Verifica que tanto el usuario como el grupo de destino existan y estén activos.

4.15. Comando MKDIR

Descripción: Crea uno o varios directorios en la jerarquía del sistema de archivos.

Parámetro	Condición	Descripción
path	Obligatorio	Ruta absoluta de la carpeta a crear.
p	Opcional	Bandera de creación recursiva de directorios padre inexistentes.

Ejemplo de uso:
mkdir -path=/home/archivos/docs -p

Efecto sobre estructuras internas: Asigna un inodo de tipo carpeta ('0') en el bitmap, crea el bloque de carpeta correspondiente con entradas '.' y '..', y actualiza la carpeta padre. Registra en Journaling si es EXT3.

Validaciones y control de errores: Verifica permisos de escritura en la carpeta padre. Falla si los padres no existen y no se especificó la bandera -p.

4.16. Comando MKFILE

Descripción: Crea un archivo nuevo con tamaño determinado o contenido provisto.

Parámetro	Condición	Descripción
path	Obligatorio	Ruta absoluta del archivo.
r	Opcional	Bandera de creación recursiva de carpetas intermedias.
size	Opcional	Tamaño en bytes (rellenado con caracteres '0'..'9'). Por defecto 0.
cont	Opcional	Ruta en el sistema anfitrión de un archivo cuyo contenido se copiará.

Ejemplo de uso:
mkfile -path=/home/archivos/nota.txt -size=25 -r

Efecto sobre estructuras internas: Reserva un inodo de tipo archivo ('1'), crea los Fileblocks necesarios (directos o indirectos), escribe los datos y actualiza la carpeta padre. Registra en Journaling si es EXT3.

Validaciones y control de errores: Valida permisos de escritura en la carpeta contenedora y disponibilidad de bloques e inodos en los bitmaps.

4.17. Comando CAT

Descripción: Despliega en pantalla el contenido textual de uno o varios archivos.

Parámetro	Condición	Descripción
fileN	Obligatorio	Rutas de los archivos a leer (ej. -file1=..., -file2=...).

Ejemplo de uso:
cat -file1=/home/archivos/nota.txt

Efecto sobre estructuras internas: Localiza el inodo del archivo, recorre sus bloques directos e indirectos, concatena las cadenas y las despliega.

Validaciones y control de errores: Exige permisos de lectura ('r') sobre el inodo del archivo. Falla si la ruta no existe o apunta a un directorio.

4.18. Comando REMOVE

Descripción: Elimina un archivo o una carpeta (y todo su contenido recursivo si aplica).

Parámetro	Condición	Descripción
path	Obligatorio	Ruta absoluta del archivo o carpeta a eliminar.

Ejemplo de uso:
remove -path=/home/archivos/docs

Efecto sobre estructuras internas: Libera los inodos y bloques en los bitmaps correspondientes y desvincula la entrada en la carpeta padre. Registra en Journaling si es EXT3.

Validaciones y control de errores: Exige permisos de escritura sobre la carpeta contenedora. Si es carpeta, elimina en cascada respetando permisos.

4.19. Comando RENAME

Descripción: Cambia el nombre de un archivo o carpeta existente.

Parámetro	Condición	Descripción
path	Obligatorio	Ruta absoluta del elemento.
name	Obligatorio	Nuevo nombre del archivo o directorio.

Ejemplo de uso:
rename -path=/home/archivos/nota.txt -name=documento.txt

Efecto sobre estructuras internas: Localiza la entrada en el Bloque de Carpetas del padre y actualiza el campo b_name. Actualiza la fecha mtime del inodo y registra en Journaling si es EXT3.

Validaciones y control de errores: Requiere permisos de escritura sobre la carpeta contenedora. Falla si ya existe un elemento con el nuevo nombre.

4.20. Comando COPY

Descripción: Realiza un duplicado exacto de un archivo o carpeta en una nueva ubicación.

Parámetro	Condición	Descripción
path	Obligatorio	Ruta absoluta del elemento origen a copiar.
destino	Obligatorio	Ruta absoluta de la carpeta de destino donde se pegará la copia.

Ejemplo de uso:
copy -path=/home/archivos/nota.txt -destino=/home/respaldos

Efecto sobre estructuras internas: Crea réplicas completas de inodos y bloques asignando nuevas posiciones en los bitmaps. Registra en Journaling si es EXT3.

Validaciones y control de errores: Requiere permisos de lectura en origen y escritura en destino. Verifica espacio libre en la partición.

4.21. Comando MOVE

Descripción: Traslada un archivo o árbol de carpetas de una ubicación a otra dentro de la partición.

Parámetro	Condición	Descripción
path	Obligatorio	Ruta del elemento origen a mover.
destino	Obligatorio	Ruta de la carpeta de destino.

Ejemplo de uso:
move -path=/home/archivos/nota.txt -destino=/home/respaldos

Efecto sobre estructuras internas: Inserta la referencia del inodo en el Bloque de Carpetas de destino y elimina la referencia en el Bloque de Carpetas original (sin duplicar bloques de datos). Registra en Journaling si es EXT3.

Validaciones y control de errores: Requiere permisos de escritura en la carpeta padre de origen y en la carpeta de destino. Falla si ya existe un elemento con el mismo nombre en destino.

4.22. Comando FIND

Descripción: Busca y lista todas las coincidencias de archivos y carpetas que concuerden con un patrón dado.

Parámetro	Condición	Descripción
path	Obligatorio	Ruta base a partir de la cual se inicia la búsqueda.
name	Obligatorio	Patrón de búsqueda con soporte para comodines '?' (un carácter) y '*' (cero o más caracteres).

Ejemplo de uso:
find -path=/home -name=*.txt

Efecto sobre estructuras internas: Recorre recursivamente los bloques de carpetas evaluando cada nombre contra la expresión de comodines y despliega los resultados con su ruta completa.

Validaciones y control de errores: Exige permisos de lectura en las carpetas exploradas. Falla si la ruta inicial no existe.

4.23. Comando CHOWN

Descripción: Cambia el usuario propietario de un archivo o directorio.

Parámetro	Condición	Descripción
path	Obligatorio	Ruta del archivo o carpeta.
user	Obligatorio	Nombre del nuevo usuario propietario.
r	Opcional	Bandera de aplicación recursiva a subdirectorios y archivos.

Ejemplo de uso:
chown -path=/home/archivos -user=operador -r

Efecto sobre estructuras internas: Modifica el campo i_uid en el inodo correspondiente asignando el UID del usuario especificado. Registra en Journaling si es EXT3.

Validaciones y control de errores: Exclusivo para el usuario 'root' o el propietario actual del archivo. Falla si el usuario indicado no existe en users.txt.

4.24. Comando CHMOD

Descripción: Modifica los permisos de acceso de un archivo o carpeta en notación octal.

Parámetro	Condición	Descripción
path	Obligatorio	Ruta del archivo o carpeta.
ugo	Obligatorio	Permisos de tres dígitos en base octal (ej. 755, 664).
r	Opcional	Bandera de aplicación recursiva.

Ejemplo de uso:
chmod -path=/home/archivos/nota.txt -ugo=755

Efecto sobre estructuras internas: Actualiza el campo i_perm en el inodo del elemento y registra en Journaling si es EXT3.

Validaciones y control de errores: Permitido únicamente al usuario 'root' o al propietario del archivo. Falla si el valor octal es inválido.
[COMPLETAR: ¿implementado? sí/no]

4.25. Comando LOSS

Descripción: Simula una pérdida catastrófica en la partición corrompiendo o sobreescribiendo sus estructuras lógicas principales.

Parámetro	Condición	Descripción
id	Obligatorio	Identificador de la partición montada a afectar.

Ejemplo de uso:
loss -id=761A

Efecto sobre estructuras internas: Rellena con caracteres nulos o ceros las zonas de los bitmaps de inodos, bitmaps de bloques, tablas de inodos y bloques, preservando el Superbloque y la zona de Journaling.

Validaciones y control de errores: Verifica que la partición esté montada. Emula la corrupción para verificar la posterior auditoría mediante Journaling.

4.26. Comando JOURNALING

Descripción: Genera una consulta o reporte en pantalla con la bitácora de operaciones registradas en EXT3.

Parámetro	Condición	Descripción
id	Obligatorio	Identificador de la partición montada (EXT3).

Ejemplo de uso:
journaling -id=761A

Efecto sobre estructuras internas: Lee secuencialmente las 50 estructuras Journal a partir del byte de inicio del journal en la partición y despliega las transacciones activas con su timestamp y contenido.

Validaciones y control de errores: Falla si la partición no está montada o si el sistema de archivos formateado es EXT2 (no posee soporte de bitácora).

4.27. Comando REP

Descripción: Genera reportes técnicos y diagramas visuales del sistema en formatos como JPG, PNG, PDF o TXT.

Parámetro	Condición	Descripción
name	Obligatorio	Tipo de reporte: 'mbr', 'disk', 'inode', 'block', 'bm_inode', 'bm_block', 'tree', 'sb', 'file', 'journaling'.
path	Obligatorio	Ruta del archivo de salida a generar en el servidor.
id	Obligatorio	Identificador de la partición montada.
ruta	Opcional	Ruta específica de archivo para reportes de tipo 'file'.

Ejemplo de uso:
rep -id=761A -path=/home/user/reports/mbr.png -name=mbr

Efecto sobre estructuras internas: Construye el código fuente en lenguaje DOT (Graphviz), invoca la utilidad 'dot' para compilar la imagen y la guarda en la ruta de destino.

Validaciones y control de errores: Verifica que la partición esté montada y crea automáticamente los directorios padre de la ruta de salida si no existen.

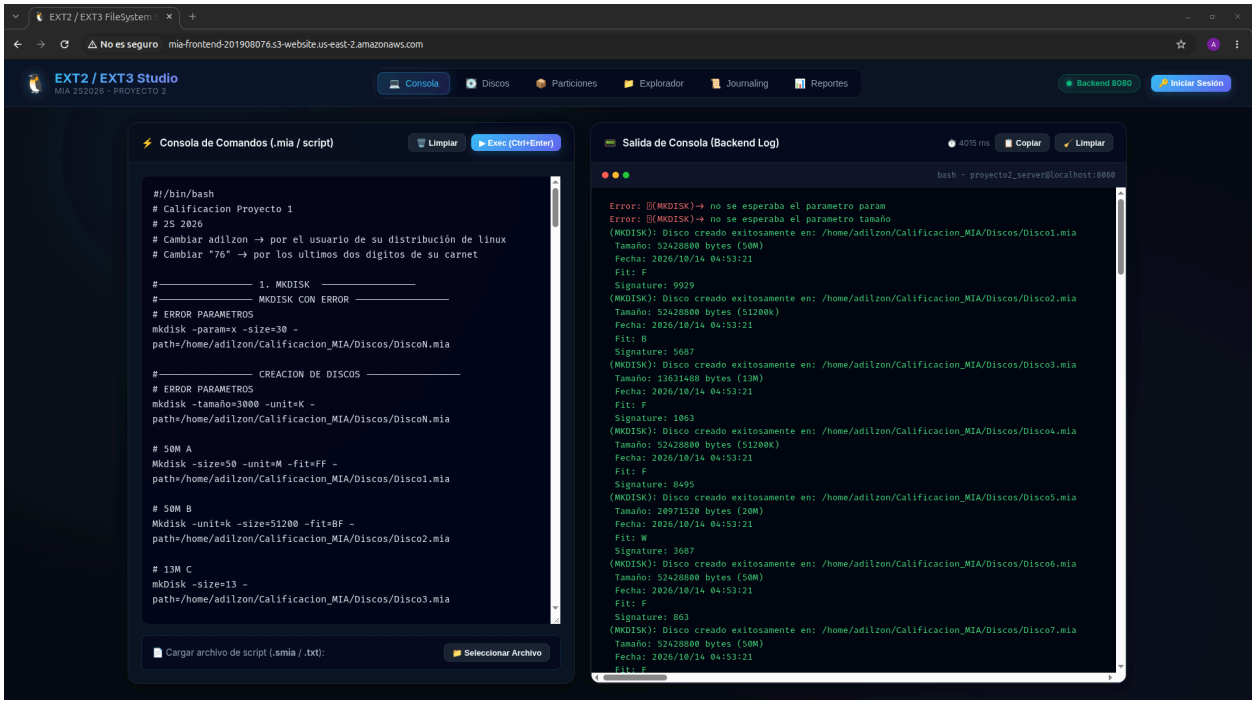


Figura 4 Ejecución y validación del comandos.

5. VISUALIZADOR DEL SISTEMA DE ARCHIVOS (INTERFAZ WEB)

El cliente web implementa un entorno gráfico reactivo que permite la inspección en tiempo real de los discos virtuales, superando las limitaciones de la interacción por consola puramente textual.

5.1. Flujo de Navegación del Usuario

- 1. Detección y Selección de Discos: El componente DiskSelector consulta el endpoint /api/disks, renderizando una cuadrícula con los discos detectados, tamaño total, firma binaria y número de particiones reconocidas.
- 2. Inspección de Particiones: Al seleccionar un disco, PartitionSelector despliega la distribución de particiones primarias, extendidas y lógicas, indicando su estado de montaje y su ID asociado.
- 3. Explorador Jerárquico: FileVisualizer permite recorrer el árbol de directorios a partir de la raíz ('/'), mostrando nombres, iconos de tipo (carpeta o archivo), tamaño en bytes, permisos octales (UGO) y propietario.
- 4. Visor de Archivos: Al hacer clic sobre un inodo de tipo archivo, se abre un panel lateral o modal que despliega el contenido textual recuperado dinámicamente desde los Fileblocks correspondientes.
- 5. Monitor de Bitácora (Journaling): JournalViewer consume el endpoint /api/journal, presentando una tabla interactiva con las últimas 50 operaciones transaccionales registradas en la partición EXT3.

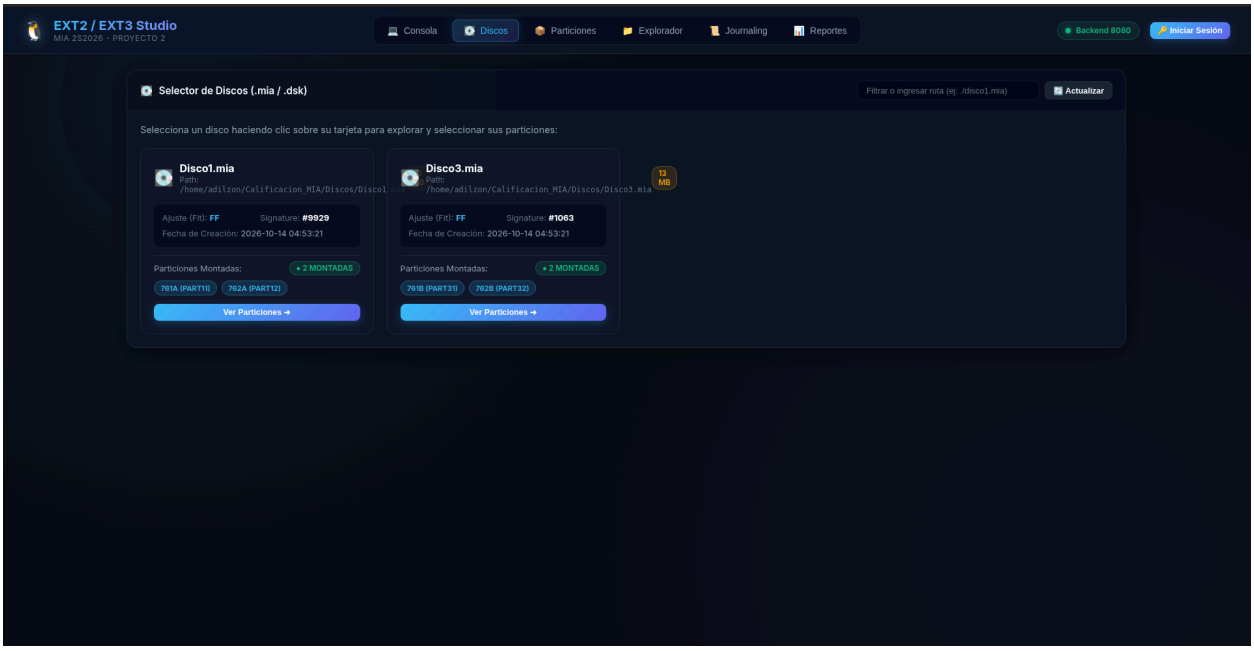


Figura 5.1. Panel principal de selección de discos virtuales y particiones en la interfaz web.

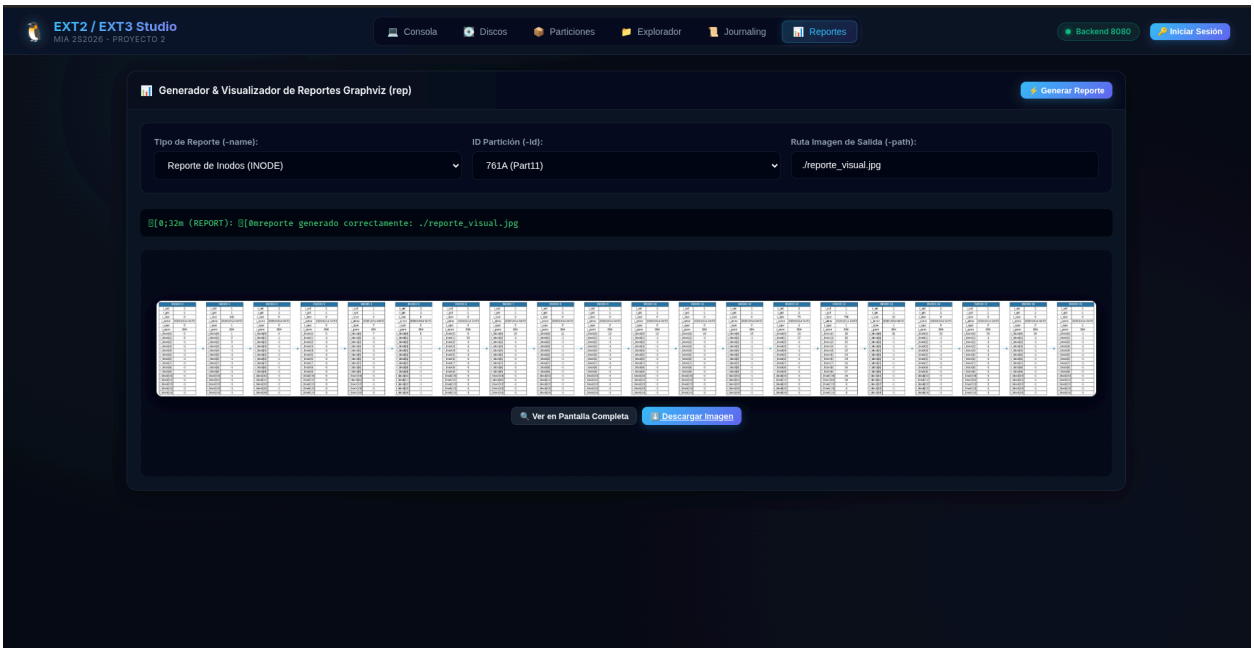


Figura 5.2. Explorador gráfico de archivos e inspección de metadatos de inodos.

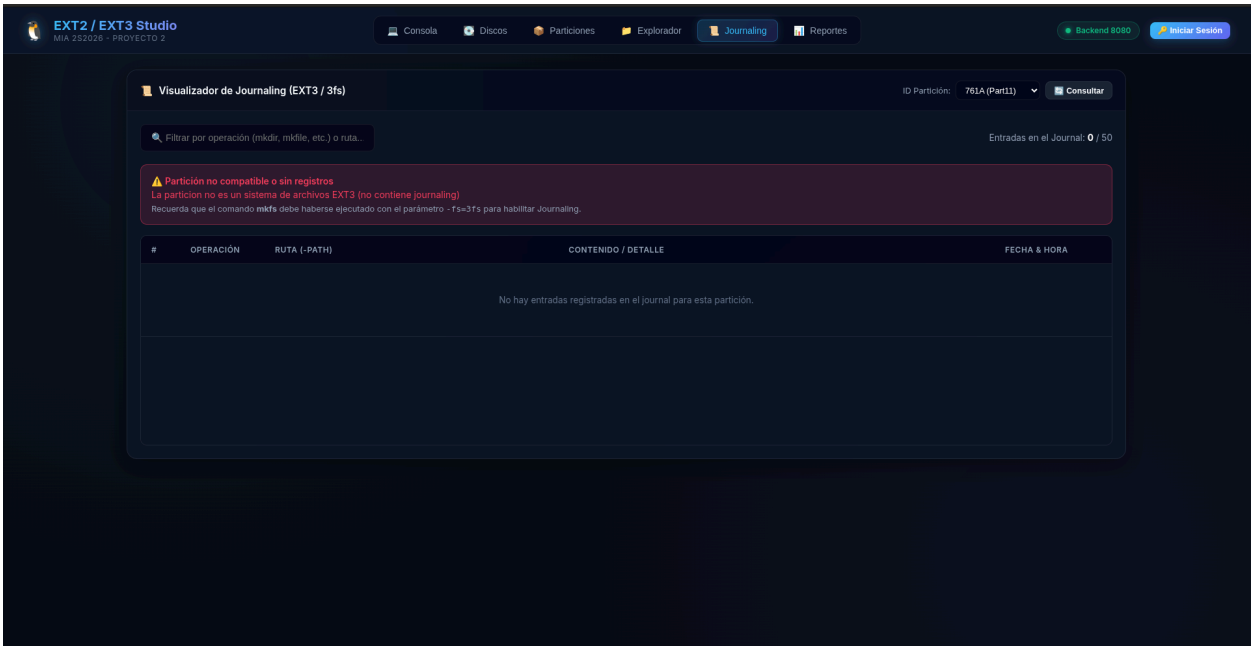


Figura 5.3. Visualizador interactivo de bitácora de Journaling en el frontend.

6. MANEJO DE PERMISOS Y SESIONES

6.1. Gestión de Usuarios y Grupos (users.txt)

El sistema implementa el control de acceso de usuarios y grupos inspirado en Unix. La información se almacena en el archivo reservado '/users.txt' (asociado al inodo 1 durante el formateo inicial).

Cada línea de users.txt sigue la sintaxis de valores separados por comas (CSV):

- Para Grupos: GID, Tipo (G), NombreGrupo
- Para Usuarios: UID, Tipo (U), NombreGrupo, NombreUsuario, Contraseña
- Ejemplo de archivo inicial tras mkfs:
  1,G,root
  1,U,root,root,123
- Eliminación lógica: Cuando un usuario o grupo es eliminado con rmusr o rmgrp, su identificador numérico se reescribe a '0', impidiendo futuras autenticaciones o asignaciones pero manteniendo la trazabilidad histórica.

6.2. Ciclo de Vida de la Sesión de Usuario

El estado de sesión es administrado en el servidor backend:

- Inicio de Sesión (Login): El comando o petición POST /api/login verifica que el usuario y contraseña coincidan con un registro de users.txt con UID > 0. Se registra el UID, GID y nombre en la sesión activa.

- Sesión Activa: Permite la ejecución de comandos con control de acceso (mkdir, mkfile, remove, rename, etc.).
- Cierre de Sesión (Logout): Limpia las credenciales en memoria restableciendo el estado no autenticado.

6.3. Matriz de Permisos Unix (UGO)

Cada inodo almacena un valor entero que representa permisos en base octal de tres dígitos (ejemplo 755 o 664):

- Primer dígito (U - User): Permisos del usuario propietario del inodo.
- Segundo dígito (G - Group): Permisos de los miembros del grupo propietario del inodo.
- Tercer dígito (O - Others): Permisos para cualquier otro usuario del sistema.
- Los dígitos se componen de la suma de bits de acceso: Lectura (r = 4), Escritura (w = 2), Ejecución/Búsqueda (x = 1).

6.4. Reglas de Validación para Operaciones del Sistema de Archivos

Operación	Regla de Autorización y Permisos Requeridos
mkdir / mkfile	Requiere permiso de escritura ('w' = 2) y ejecución ('x' = 1) sobre el inodo de la carpeta contenedora inmediata.
cat	Requiere permiso de lectura ('r' = 4) sobre el inodo del archivo, y ejecución ('x') en todos los directorios que componen la ruta.
remove	Requiere permiso de escritura ('w') sobre la carpeta padre para desvincular la entrada de directorio.
rename	Requiere permiso de escritura ('w') sobre la carpeta padre para actualizar el nombre de la entrada de directorio.
copy	Requiere permiso de lectura ('r') sobre el elemento de origen y permiso de escritura ('w') sobre la carpeta de destino.
move	Requiere permiso de escritura ('w') sobre la carpeta padre de origen (para remover la entrada) y sobre la carpeta padre de destino (para registrarla).
chown	Operación restringida exclusivamente al usuario 'root' (UID 1) o al propietario actual del inodo.

chmod	Operación restringida exclusivamente al usuario 'root' (UID 1) o al propietario actual del inodo.

```cpp
bool FileManager::check_permission(const Structs::Inodes &inode, const Users::User &logged, int req_perm) {
    if (logged.uid == 1 || shared.compare(logged.user, "root")) {
        return true;
    }

    int p = inode.i_perm;
    int u_perm = (p / 100) % 10;
    int g_perm = (p / 10) % 10;
    int o_perm = p % 10;

    int perm_level = o_perm;
    if (inode.i_uid == logged.uid) {
        perm_level = u_perm;
    } else if (inode.i_gid == logged.gid) {
        perm_level = g_perm;
    }

    // req_perm: 4 = R, 2 = W, 1 = X
    return (perm_level & req_perm) == req_perm;
}
```

Listado 6.1. Algoritmo de validación de permisos UGO en C++.

7. PÉRDIDA DEL SISTEMA (LOSS) Y JOURNALING

7.1. Simulación de Pérdida del Sistema (loss)

El comando 'loss' emula un evento de falla catastrófica de hardware o corrupción en disco que inutiliza el sistema de archivos tradicional:

- Estructuras Afectadas: Se sobreescriben con ceros los sectores correspondientes al Bitmap de Inodos, Bitmap de Bloques, la Tabla de Inodos y la Tabla de Bloques de datos.
- Estructuras Preservadas: El Superbloque se mantiene íntegro, así como la zona fija de Journaling (50 estructuras Journal contiguas al Superbloque).
- Efecto resultante: Cualquier intento de navegación regular reporta el sistema de archivos como vacío o no legible.

```
void FileManager::loss_cmd(vector<string> context, Mount &mount_inst) {
   try {
       string id = "";
       for (auto current : context) {
           string key = shared.lower(current.substr(0,
current.find('=')));
           if (shared.compare(key, "id")) {
               current.erase(0, key.length() + 1);
               if (current.front() == '\"') current = current.substr(1,
current.length() - 2);
               id = current;
           }
       }

       if (id.empty()) throw runtime_error("requiere parámetro
obligatorio -id");

       string disk_path;
       Structs::Partition partition = mount_inst.getmount(id,
&disk_path);
       if (partition.part_start == -1) throw runtime_error("partición
no montada: " + id);

       FILE *disk = fopen(disk_path.c_str(), "rb+");
```

```
        if (disk == NULL) throw runtime_error("no se pudo abrir el
disco");

        Structs::Superblock sb;
        fseek(disk, partition.part_start, SEEK_SET);
        if (fread(&sb, sizeof(Structs::Superblock), 1, disk) != 1) {
            fclose(disk);
            throw runtime_error("error al leer el superbloque");
        }

        if (sb.s_filesystem_type != 3) {
            fclose(disk);
            throw runtime_error("el comando loss solo aplica a sistemas
EXT3");
        }

        // Mostrar estado previo
        cout << "\n[LOSS] Iniciando simulación de pérdida en partición "
<< id << "..." << endl;
        cout << "[LOSS] Bloques ocupados previos: " <<
(sb.s_blocks_count - sb.s_free_blocks_count) << endl;
        cout << "[LOSS] Inodos ocupados previos: " << (sb.s_inodes_count
- sb.s_free_inodes_count) << endl;

        // Limpiar bitmaps con '0' y áreas de inodos y bloques con '\0'
        int bm_inodes_size = sb.s_inodes_count;
        int bm_blocks_size = sb.s_blocks_count;
        int inodes_table_size = sb.s_inodes_count *
sizeof(Structs::Inodes);
        int blocks_table_size = sb.s_blocks_count *
sizeof(Structs::Folderblock);

        // 1. Limpiar Bitmap de Inodos
        fseek(disk, sb.s_bm_inode_start, SEEK_SET);
        vector<char> zero_char(bm_inodes_size, '0');
```

```
        fwrite(zero_char.data(), 1, bm_inodes_size, disk);

        // 2. Limpiar Bitmap de Bloques
        fseek(disk, sb.s_bm_block_start, SEEK_SET);
        vector<char> zero_blocks(bm_blocks_size, '0');
        fwrite(zero_blocks.data(), 1, bm_blocks_size, disk);

        // 3. Limpiar Área de Inodos
        fseek(disk, sb.s_inode_start, SEEK_SET);
        vector<char> zero_null(65536, '\0');
        int remaining = inodes_table_size;
        while (remaining > 0) {
            int to_w = min(remaining, (int)zero_null.size());
            fwrite(zero_null.data(), 1, to_w, disk);
            remaining -= to_w;
        }

        // 4. Limpiar Área de Bloques
        fseek(disk, sb.s_block_start, SEEK_SET);
        remaining = blocks_table_size;
        while (remaining > 0) {
            int to_w = min(remaining, (int)zero_null.size());
            fwrite(zero_null.data(), 1, to_w, disk);
            remaining -= to_w;
        }

        fclose(disk);
        shared.response("LOSS", "pérdida del sistema de archivos
simulada exitosamente (-id=" + id + ")");
        cout << "[LOSS] El Journaling permanece intacto para
recuperación futura.\n" << endl;
    }
    catch (exception &e) {
        shared.handler("LOSS", e.what());
```

```
    }
}
```

Listado 7.1. Sobreescritura de sectores críticos en el comando loss.

7.2. Registro y Estructura de Transacciones en EXT3

Para mitigar la pérdida de consistencia, cada operación atómica que altera el sistema de archivos (mkdir, mkfile, remove, rename, etc.) se registra en la bitácora circular antes o inmediatamente después de efectuarse en las estructuras secundarias:

- Capacidad del Journal: Constante fija de 50 entradas (Structs::Journal).
- Campos de Transacción: Se guardan el número correlativo de operación (j_count), el nombre del comando (i_operation), la ruta afectada (i_path), los datos adjuntos (i_content) y la marca de tiempo (i_date).

7.3. Auditoría y Visualización en la Interfaz Gráfica

A través del comando 'journaling' o desde la pestaña Journal de la interfaz web, los administradores pueden auditar la totalidad de modificaciones cronológicas realizadas, incluso después de un evento 'loss', permitiendo una reconstrucción o auditoría completa del historial del disco.

8. ERRORES CONOCIDOS, LIMITACIONES Y DECISIONES TÉCNICAS

8.1. Errores Conocidos y Comportamientos Inesperados

- [COMPLETAR: Documentar si existe algún error conocido al ejecutar scripts con rutas con espacios o caracteres especiales]
- [COMPLETAR: Documentar posibles problemas de visualización al renderizar reportes Graphviz de gran tamaño]
- [COMPLETAR: Describir si ocurre alguna inconsistencia al desmontar un disco con sesión activa abierta]

8.2. Limitaciones Técnicas del Sistema

- Límite de Particiones: Máximo 4 particiones primarias/extendidas por disco MBR según estándar tradicional.
- Capacidad de la Bitácora: El Journaling posee una capacidad de tamaño fijo delimitada a [COMPLETAR: 50 entradas fijas / circular].
- Direccionamiento Indirecto: [COMPLETAR: Especificar si se implementó direccionamiento indirecto simple, doble y triple en la lectura/escritura de archivos extensos].

- Concurrencia de Peticiones: El servidor cpp-httplib procesa peticiones en [COMPLETAR: un solo hilo / multi-hilo con mutex de disco] sobre los archivos binarios.

8.3. Decisiones de Diseño e Implementación

- Selección de Biblioteca HTTP: Se eligió cpp-httplib sobre alternativas basadas en sockets POSIX directos por su facilidad para gestionar cabeceras CORS de forma homogénea y serialización JSON directa.
- Almacenamiento de Discos en el Servidor: Los archivos binarios .mia se persisten en [COMPLETAR: ruta de almacenamiento en el servidor, ej. directorio de ejecución ./disks].
- Estrategia de Montaje en Memoria: La tabla de montajes se mantiene en memoria volátil de la instancia backend, lo que optimiza la velocidad pero requiere volver a montar las particiones tras reiniciar el proceso api_server.

9. CONCLUSIONES

- La simulación de sistemas de archivos EXT2 y EXT3 en lenguaje C++ demuestra cómo las estructuras de bajo nivel, como el MBR, el superbloque, los inodos y los bloques de datos, interactúan para organizar y administrar la información almacenada en los discos virtuales. La implementación de estas estructuras permite comprender cómo se distribuyen los datos dentro de una partición y cómo se facilita su localización, indexación y persistencia en la memoria secundaria.
- El mecanismo de *Journaling* implementado en EXT3 aporta una capa adicional de integridad al registrar las operaciones en una bitácora antes de que se reflejen completamente en las estructuras del sistema de archivos. Este mecanismo facilita la auditoría de las operaciones realizadas y contribuye a la recuperación de información ante fallas simuladas mediante el comando `loss`, permitiendo analizar la importancia de mantener un registro de las transacciones para reducir el impacto de posibles errores.
- La arquitectura desacoplada, basada en servicios REST, permitió integrar un motor de procesamiento desarrollado en C++ y desplegado en AWS EC2 con una interfaz web alojada en AWS S3. Esta separación facilita la comunicación entre la interfaz gráfica y la lógica encargada de administrar los discos virtuales, ejecutar comandos y consultar información del sistema de archivos. De esta manera, el proyecto demuestra la viabilidad de integrar una aplicación de bajo nivel con tecnologías web y servicios en la nube, manteniendo separadas las responsabilidades de cada componente.
- La implementación del modelo de permisos Unix (UGO), junto con la administración de usuarios y grupos mediante el archivo `/users.txt`, permite establecer un control de acceso sobre los archivos y directorios del sistema simulado. Mediante los permisos de lectura, escritura y ejecución, se definen las operaciones que cada usuario puede realizar según sus privilegios. Esto permite comprender la importancia de la autenticación, la asignación de permisos y la administración de usuarios y grupos para mantener un entorno organizado y protegido frente a operaciones no autorizadas.